ITELEC 4100
Lec # 3

Introduction to React JS

1

REACT JS

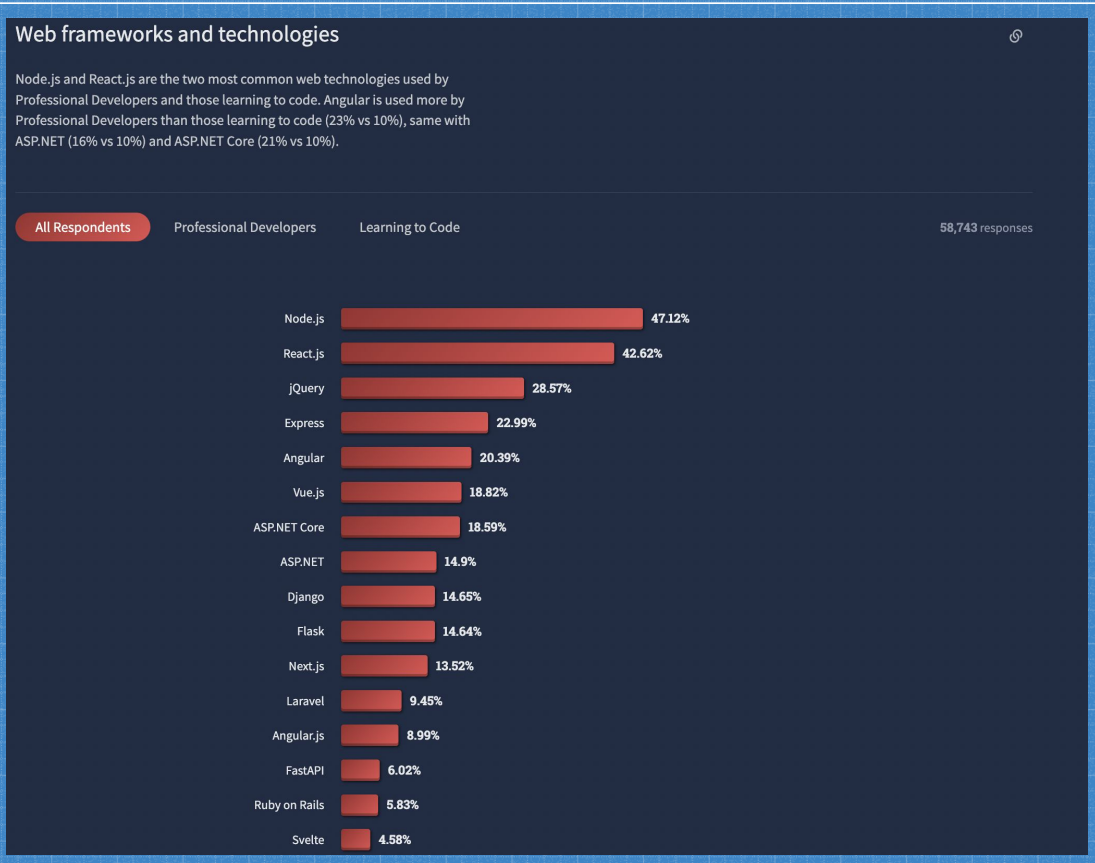
What is React?

- Open source library for building user interfaces
- Focus on UI
- Rich ecosystem
- Created and maintained by Meta (Facebook)

Why learn React?

- More than a 100,000 starts git files on Github
- Huge community
- Popular and in demand skill set.

Why learn React?



React Key point

- Component Based Architecture
- Reusable code
- Declarative paradigm
  - Tell React what you want and React will build the actual UI.

Prerequisites

- HTML
- CSS
- JavaScript fundamentals
  - ‘this’ keyword, filter, map
- ES6
  - let & const
  - Arrow functions
  - Template literals
  - Default parameters
  - Object literals
  - Rest and spread operators

Hyper Text Markup Language (HTML)

- Html is the standard markup language for documents designed to be displayed in a web browser.

Cascading Style Sheet (CSS)

- CSS is a style sheet language used for describing the presentation of a document written in a markup language such as HTML and XML.

JavaScript (JS)

- JS is a scripting language that enables you to create dynamically updating content for both HTML and CSS.
- JS can also calculate, manipulate and validate data.

ECMAScript 2015 (6th version - ES6)

- ES6 is a standardization for creating a scripting language.
- It was introduced by Ecma International.
- It is basically an implementation that tells us how to create/use a scripting language.
- JavaScript conforms to the ECMAScript specification.

JavaScript ES6

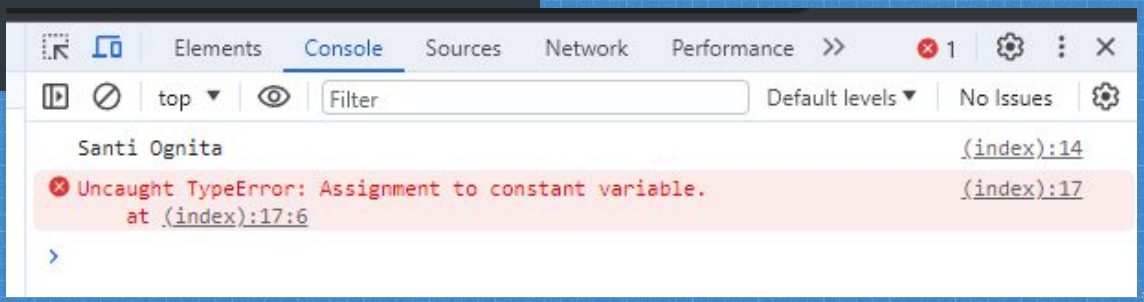
- It allows us to write a less code and in a elegant way which makes the code more modern and readable.

JavaScript ES6

- **`const`** – used to store the variable value that is not going to be changed **`except`** when used with object.
- **`let`** – value can be change like **`var`** keyword but more features like scoping which make it better compared to **`var.`**

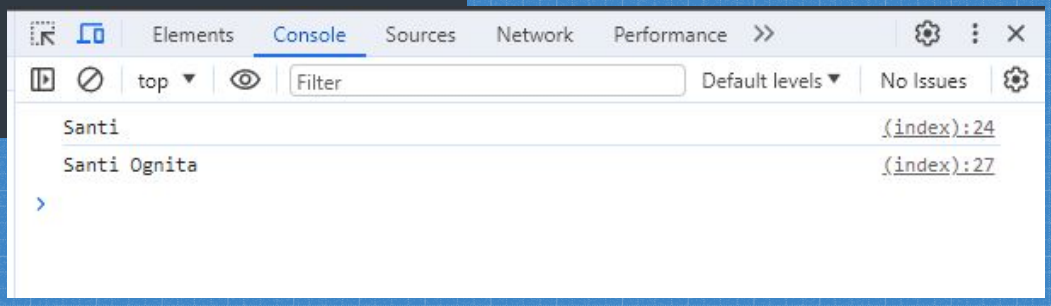
JavaScript ES6 (const sample)

```
<script type="text/javascript">

const name = 'Santi Ognita';
console.log(name);

name = 'Santi';
console.log(name);

</script>
```



JavaScript ES6 (let sample)

```
var x = "Santi Ognita";

if (true) {
  let x = "Santi";
  console.log(x);
}

console.log(x);
```



JavaScript ES6

- **`arrow function (fat arrow function)`** – a more concise syntax for writing a function expressions.

JavaScript ES6 (arrow function)

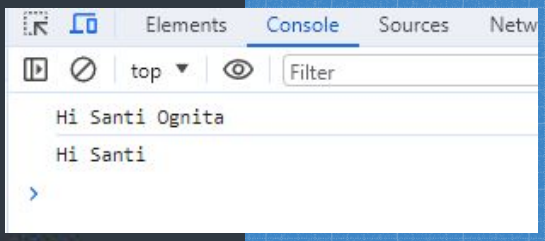
- common function in JavaScript

```
function printName(name){
    console.log('Hello '+name);
}

printName('Santi Ognita');
```


JavaScript ES6 (arrow function)

- fat arrow function

```
//sample 1
const printName = name =>{
    return `Hi ${name}`;
}
console.log(printName('Santi Ognita'));

//sample 2
const printName1 = name => `Hi ${name}`;
console.log(printName1('Santi'));
```

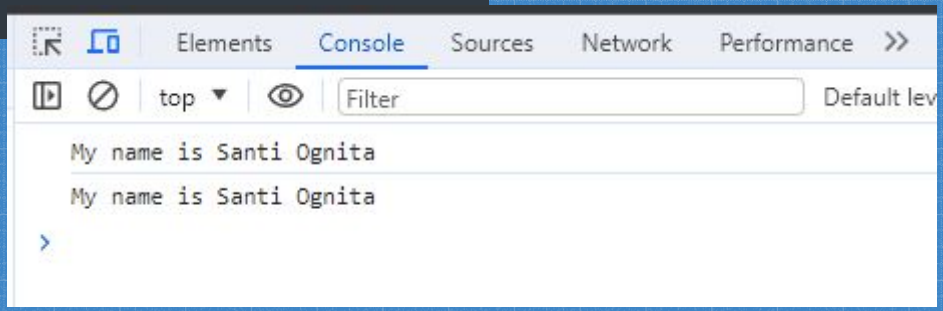


JavaScript ES6

- **`Template literals`** - allow us to work with strings in a better way.
- It is use backticks (``) rather than single (‘’) or double (“”) quotes we’ve used with strings.

JavaScript ES6 (template literals)

```
var name1 = 'Santi Ognita';
console.log('My name is '+ name1);

const name2 = 'Santi Ognita';
console.log(`My name is ${name2}`);
```

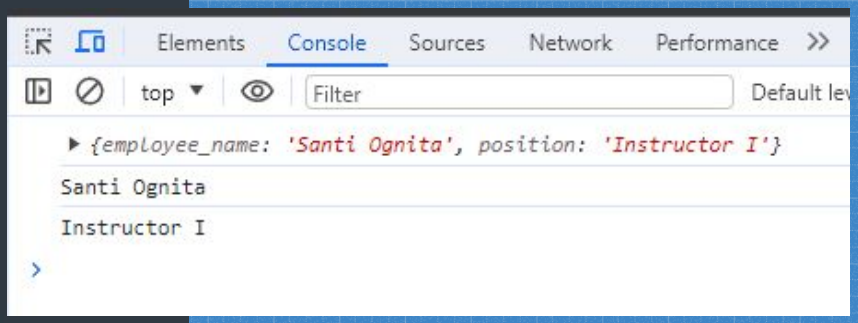


JavaScript ES6

- Object literal – makes object more concise and powerful by extending the syntax is some ways.

JavaScript ES6 (object literal)

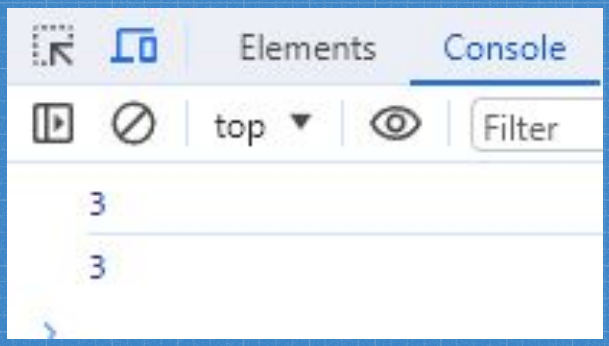
```
let name = 'employee_name';
let position = 'Instructor I'
let Employee = {
    [name]: 'Santi Ognita',
    'position': position,
};
console.log(Employee);
console.log(Employee[name]);
console.log(Employee['position']);
```



JavaScript ES6

- Default parameters – allow us to define a parameter in advance.

```
function add_num(a,b=1){
    return a + b;
}
console.log(add_num(2,1));
console.log(add_num(2));
```

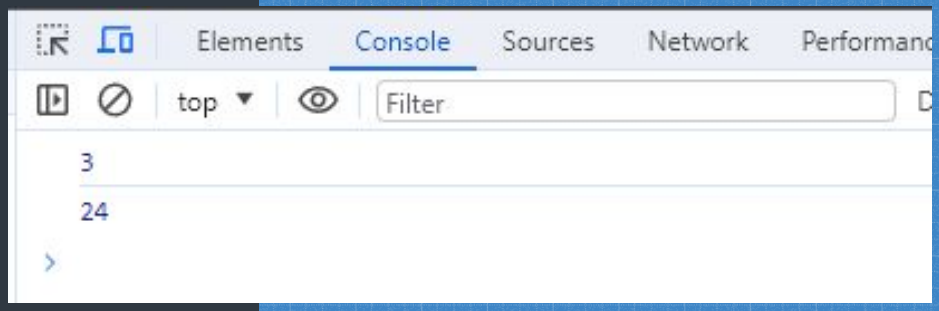


JavaScript ES6

- Rest parameters – allow us to represent an **`indefinite`** number of arguments as an array.
- Uses three dots (…) followed by the argument name.

JavaScript ES6 (rest parameter)

```
function add_num(...num){
    let sum = 0;
    for(let i of num){
        sum+=i;
    }
    return sum;
}
console.log(add_num(1,2));
console.log(add_num(1,2,3,5,6,7));
```

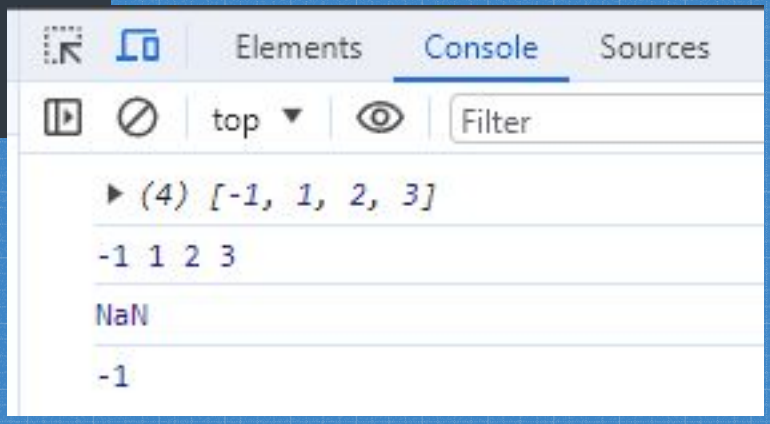


JavaScript ES6

- Spread operator – allow us the privilege to obtain a list of parameter from an array.
- Uses three dots (…) followed by the array name.

JavaScript ES6 (spread operator)

```
let arr = [-1,1,2,3];

console.log(arr);
console.log(...arr);

console.log(Math.min(arr));
console.log(Math.min(...arr));
```

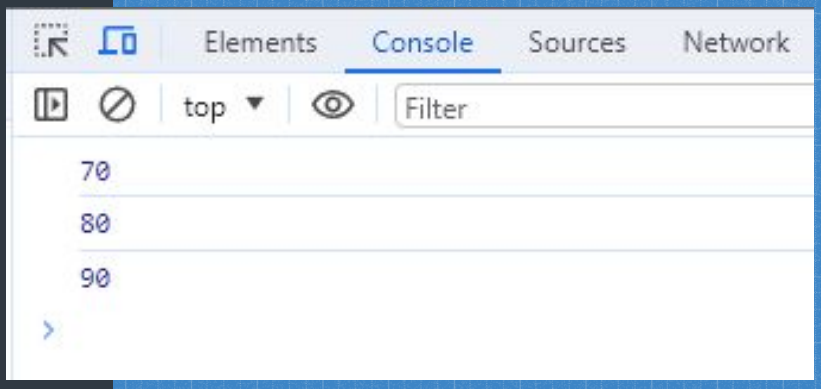


JavaScript ES6

- Destructing assignment – allows us to get each value of an array into individual variables.

JavaScript ES6(destructuring assignment)

```
let scores = [70, 80, 90];

// let x = scores[0],
//     y = scores[1],
//     z = scores[2];

let [x, y, z] = scores;

console.log(x);
console.log(y);
console.log(z);
```



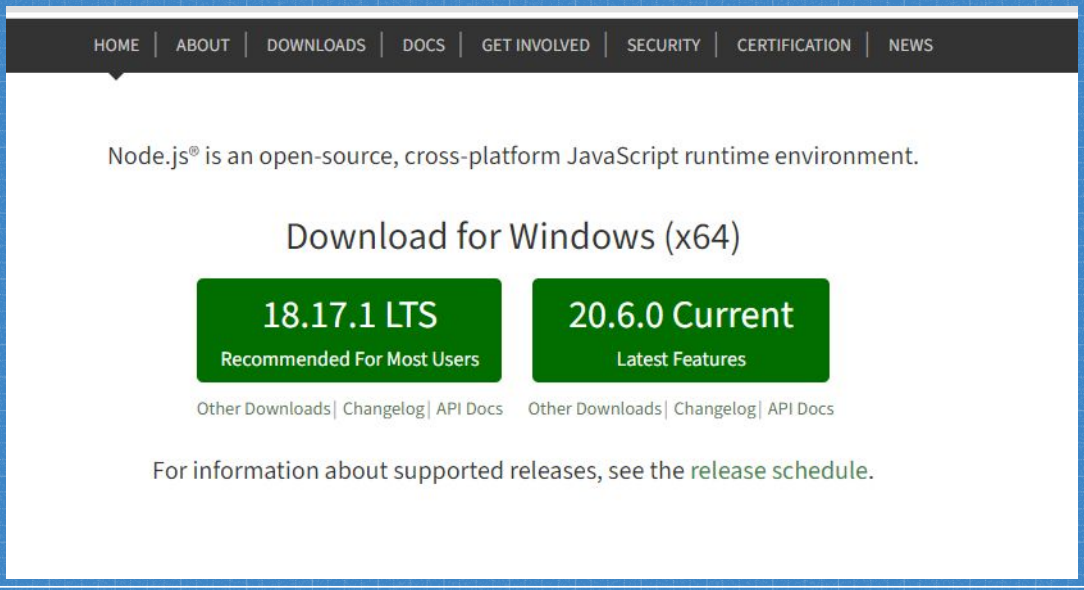
Now we are ready to learn React JS!

Getting started with React

- Text Editor (sublime, vscode)
- Install Node.js

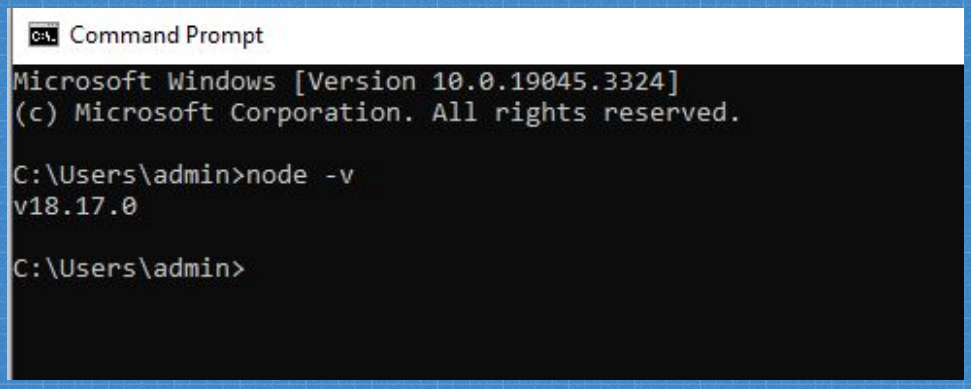
Node.js

- Link : https://nodejs.org/en



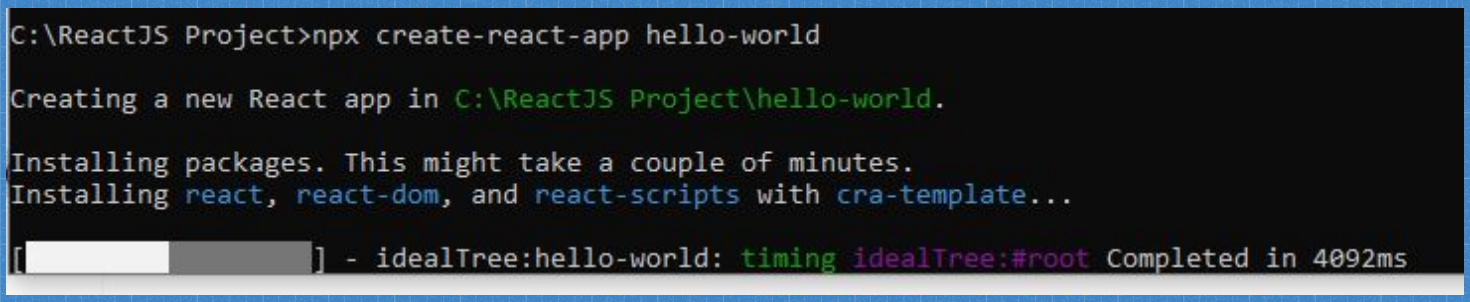
Node.js

- Check if install properly
- Run command prompt
- Type “node –v”

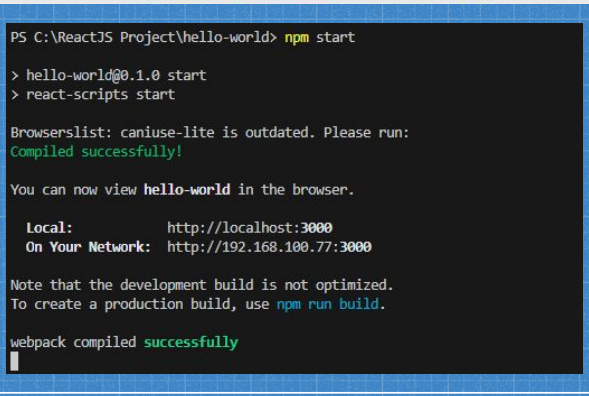


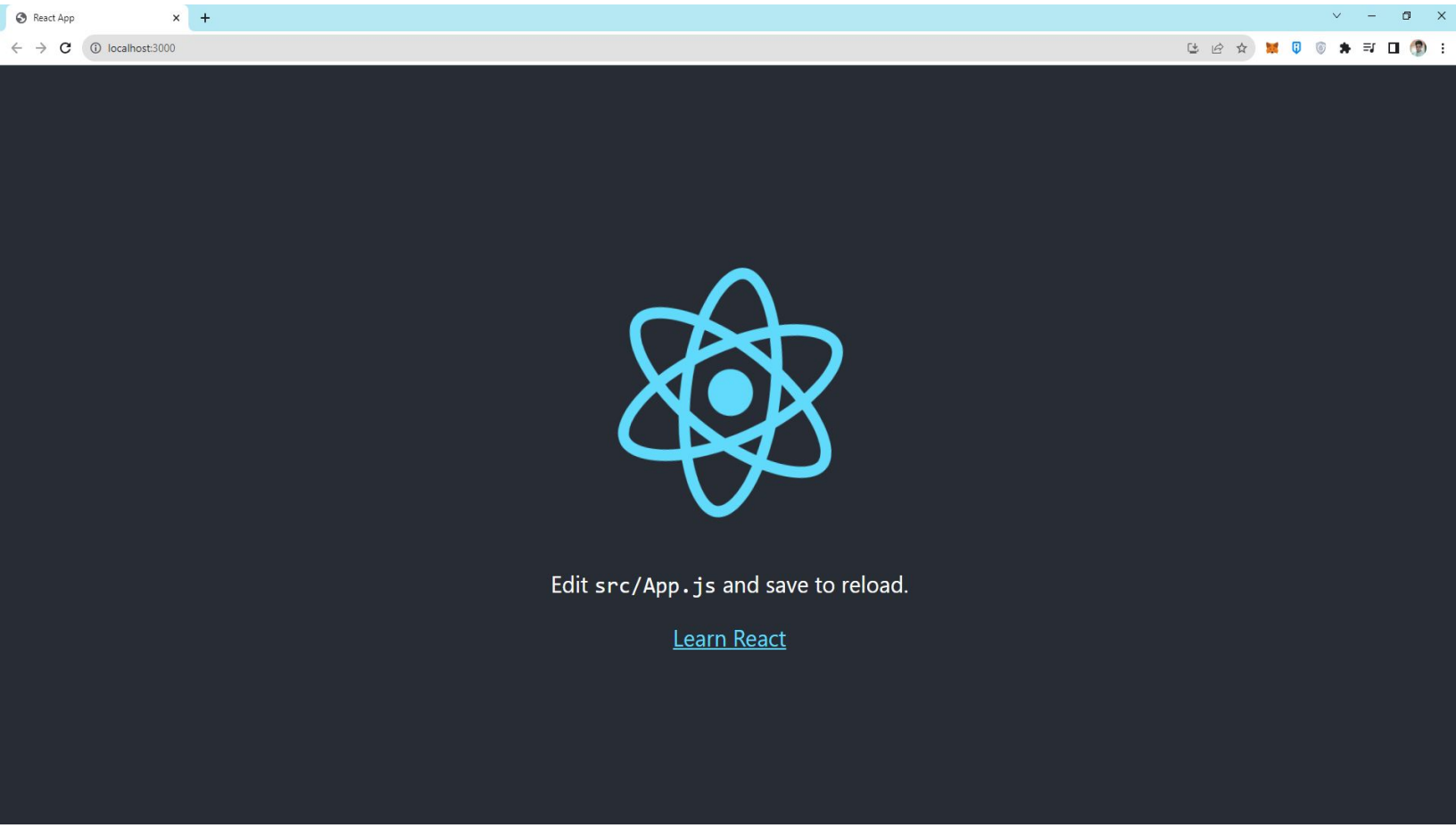
Creating a react application

- Run “npx create-react-app <app_name>”



- Run “npm start”





Components

- Functional Components
- Class Components

Functional Component

Hello.js

```
1  import React from 'react'
2
3  function Hello(){
4      return <h1>Hello World!</h1>
5  }
6
7  export default Hello
8
9
```


Class Component

Welcome.js

```
1  import React, { Component } from 'react'
2
3  class Welcome extends Component {
4      render(){
5          return <h2>Welcome to React JS</h2>
6      }
7  }
8
9  export default Welcome
10
```

Functional vs Class components

- Simple
- Should use as much as possible
- Solution without state
- Mainly responsible for the UI

- More feature rich
- Maintain their own private data (state)
- Complex UI logic
- Provide lifecycle hooks

JavaScript XML (JSX)

- Extension to the JavaScript language syntax.
- Write XML-like code for elements and components
- JSX tags have a tag name, attributes, and children.

JavaScript XML (JSX)

Note:

- JSX is not required to write React application
- It makes our code simpler and elegant.

with JSX

Hello.js

```
1  import React from 'react'
2
3  // function Hello(){
4  //  return <h1>Hello World!</h1>
5  // }
6
7  // const Hello = () => <h1>Hello World! Again?</h1>
8  const Hello = () => {
9      return (
10         <div id="id_no" className="class-name">
11             <h1>Hello JSX!</h1>
12         </div>
13         )
14 }
15
16 export default Hello
17
```

without JSX

Hello.js

```
1  import React from 'react'
2
3  // function Hello(){
4  //  return <h1>Hello World!</h1>
5  // }
6
7  // const Hello = () => <h1>Hello World! Again?</h1>
8  const Hello = () => {
9      // return (
10     //  <div id="id_no" className="class-name">
11     //      <h1>Hello JSX!</h1>
12     //  </div>
13     //  )
14     return React.createElement(
15         'div',
16         {id:"id_no",className:"class-name"},
17         React.createElement('h1',null,"No JSX :("),
18         )
19 }
20
21 export default Hello
```


Props vs State

Use of props

App.js

```
1  import logo from './logo.svg';
2  import './App.css';
3  import Hello from './components/Hello'
4  import Welcome from './components/Welcome'
5
6  function App() {
7    return (
8      <div className="App">
9        <Hello name="Santi S. Ognita" groups="MISO" />
10       <Hello name="Students" groups="BSIT 4"/>
11       <Welcome />
12     </div>
13   );
14 }
15
16 export default App;
17
```


Use of props

Hello.js

```
1  import React from 'react'
2
3  // function Hello(){
4  //  return <h1>Hello World!</h1>
5  // }
6
7  // const Hello = () => <h1>Hello World! Again?</h1>
8  const Hello = props => {
9      return (
10         <div id="id_no" className="class-name">
11             <h1>Hello {props.name} from {props.groups}</h1>
12         </div>
13         )
14     // return React.createElement(
15     //  'div',
16     //  {id:"id_no",className:"class-name"},
17     //  React.createElement('h1',null,"No JSX :("),
18     //  )
19 }
20
21 export default Hello
```

Use of state

```
import React, { Component } from 'react'

class Welcome extends Component {

    constructor(){
        super()
        this.state = {
            message:'Welcome to React JS',
        }
    }

    changeMessage(){
        this.setState({
            message:'Getting hang with React JS'
        },
            () => console.log(this.state.message)
        )
        console.log(this.state.message)
    }

    render(){
        const {message} = this.state
        return(
            <div>
                <h2>{message}</h2>
                <button onClick={ () => this.changeMessage() }>Click me!</button>
            </div>
            )
    }
}

export default Welcome
```

```
Welcome to React JS                    Welcome.js:22
Getting hang with React JS             Welcome.js:21
```


Props vs State

- Get passed to the component
- Function parameter
- Immutable

- Manage within the component
- Variable declared in the function body
- Can be changed

END.